

1622. Fancy Sequence

Solved

Hard

Topics

Companies

Hint

Write an API that generates fancy sequences using the `append`, `addAll`, and `multAll` operations.

Implement the `Fancy` class:

- `Fancy()` Initializes the object with an empty sequence.
- `void append(val)` Appends an integer `val` to the end of the sequence.
- `void addAll(inc)` Increments all existing values in the sequence by an integer `inc`.
- `void multAll(m)` Multiplies all existing values in the sequence by an integer `m`.
- `int getIndex(idx)` Gets the current value at index `idx` (0-indexed) of the sequence **modulo** $10^9 + 7$. If the index is greater or equal than the length of the sequence, return `-1`.

Example 1:

Input

```
["Fancy", "append", "addAll", "append", "multAll", "getIndex",  
"addAll", "append", "multAll", "getIndex", "getIndex", "getIndex"]  
[[], [2], [3], [7], [2], [0], [3], [10], [2], [0], [1], [2]]
```

Output

```
[null, null, null, null, null, 10, null, null, null, 26, 34, 20]
```

Explanation

```
Fancy fancy = new Fancy();  
fancy.append(2);    // fancy sequence: [2]  
fancy.addAll(3);    // fancy sequence: [2+3] -> [5]  
fancy.append(7);    // fancy sequence: [5, 7]  
fancy.multAll(2);   // fancy sequence: [5*2, 7*2] -> [10, 14]  
fancy.getIndex(0);  // return 10  
fancy.addAll(3);    // fancy sequence: [10+3, 14+3] -> [13, 17]  
fancy.append(10);   // fancy sequence: [13, 17, 10]  
fancy.multAll(2);   // fancy sequence: [13*2, 17*2, 10*2] -> [26, 34, 20]  
fancy.getIndex(0);  // return 26  
fancy.getIndex(1);  // return 34  
fancy.getIndex(2);  // return 20
```

Constraints:

- $1 \leq \text{val}, \text{inc}, m \leq 100$
- $0 \leq \text{idx} \leq 10^5$
- At most 10^5 calls total will be made to `append`, `addAll`, `multAll`, and `getIndex`.

Python:

class Fancy:

```
def __init__(self):
    self.mod = 10**9 + 7
    self.val = []
    self.a = 1
    self.b = 0

def append(self, val: int) -> None:
    x = (val - self.b + self.mod) % self.mod
    self.val.append(x * pow(self.a, self.mod - 2, self.mod) % self.mod)

def addAll(self, inc: int) -> None:
    self.b = (self.b + inc) % self.mod

def multAll(self, m: int) -> None:
    self.a = (self.a * m) % self.mod
    self.b = (self.b * m) % self.mod

def getIndex(self, idx: int) -> int:
    if idx >= len(self.val):
        return -1
    return (self.a * self.val[idx] + self.b) % self.mod
```

Your Fancy object will be instantiated and called as such:

```
# obj = Fancy()
# obj.append(val)
# obj.addAll(inc)
# obj.multAll(m)
# param_4 = obj.getIndex(idx)
```

JavaScript:

```
/**
 * @description Fancy sequence implementation using BigInt for precision
 * and modular inverse for O(1) updates.
```

```

*/
var Fancy = function() {
    this.mod = BigInt(10**9 + 7);
    this.values = [];
    this.a = 1n;
    this.b = 0n;
};

/** * Helper: Modular Exponentiation (Fast Power)
 * Computes (base^exp) % mod
 */
Fancy.prototype.power = function(base, exp) {
    let res = 1n;
    base %= this.mod;
    while (exp > 0n) {
        if (exp % 2n === 1n) res = (res * base) % this.mod;
        base = (base * base) % this.mod;
        exp /= 2n;
    }
    return res;
};

/** * Helper: Modular Inverse using Fermat's Little Theorem
 * Since mod is prime, a^(mod-2) % mod is the inverse.
 */
Fancy.prototype.modInverse = function(n) {
    return this.power(n, this.mod - 2n);
};

/** * @param {number} val
 * @return {void}
 */
Fancy.prototype.append = function(val) {
    let valBI = BigInt(val);
    let invA = this.modInverse(this.a);
    let x = ((valBI - this.b + this.mod) % this.mod * invA) % this.mod;
    this.values.push(x);
};

/** * @param {number} inc
 * @return {void}
 */
Fancy.prototype.addAll = function(inc) {
    this.b = (this.b + BigInt(inc)) % this.mod;
};

```

```

};

/** * @param {number} m
 * @return {void}
 */
Fancy.prototype.multAll = function(m) {
    let mBI = BigInt(m);
    this.a = (this.a * mBI) % this.mod;
    this.b = (this.b * mBI) % this.mod;
};

/** * @param {number} idx
 * @return {number}
 */
Fancy.prototype.getIndex = function(idx) {
    if (idx >= this.values.length)
        return -1;
    let result = (this.a * this.values[idx] + this.b) % this.mod;
    return Number(result);
};

```

Java:

```

class Fancy {
    private static final int MOD = 1000000007;
    private ArrayList<Long> val;
    private long a, b;

    public Fancy() {
        val = new ArrayList<>();
        a = 1;
        b = 0;
    }

    private long modPow(long x, long y, long mod) {
        long res = 1;
        x = x % mod;
        while (y > 0) {
            if (y % 2 == 1) {
                res = (res * x) % mod;
            }
            y = y / 2;
            x = (x * x) % mod;
        }
        return res;
    }
}

```

```

    }

    public void append(int val) {
        long x = (val - b + MOD) % MOD;
        this.val.add((x * modPow(a, MOD - 2, MOD)) % MOD);
    }

    public void addAll(int inc) {
        b = (b + inc) % MOD;
    }

    public void multAll(int m) {
        a = (a * m) % MOD;
        b = (b * m) % MOD;
    }

    public int getIndex(int idx) {
        if (idx >= val.size()) return -1;
        return (int)((a * val.get(idx) + b) % MOD);
    }
}

```

/**

* Your Fancy object will be instantiated and called as such:

* Fancy obj = new Fancy();

* obj.append(val);

* obj.addAll(inc);

* obj.multAll(m);

* int param_4 = obj.getIndex(idx);

*/